

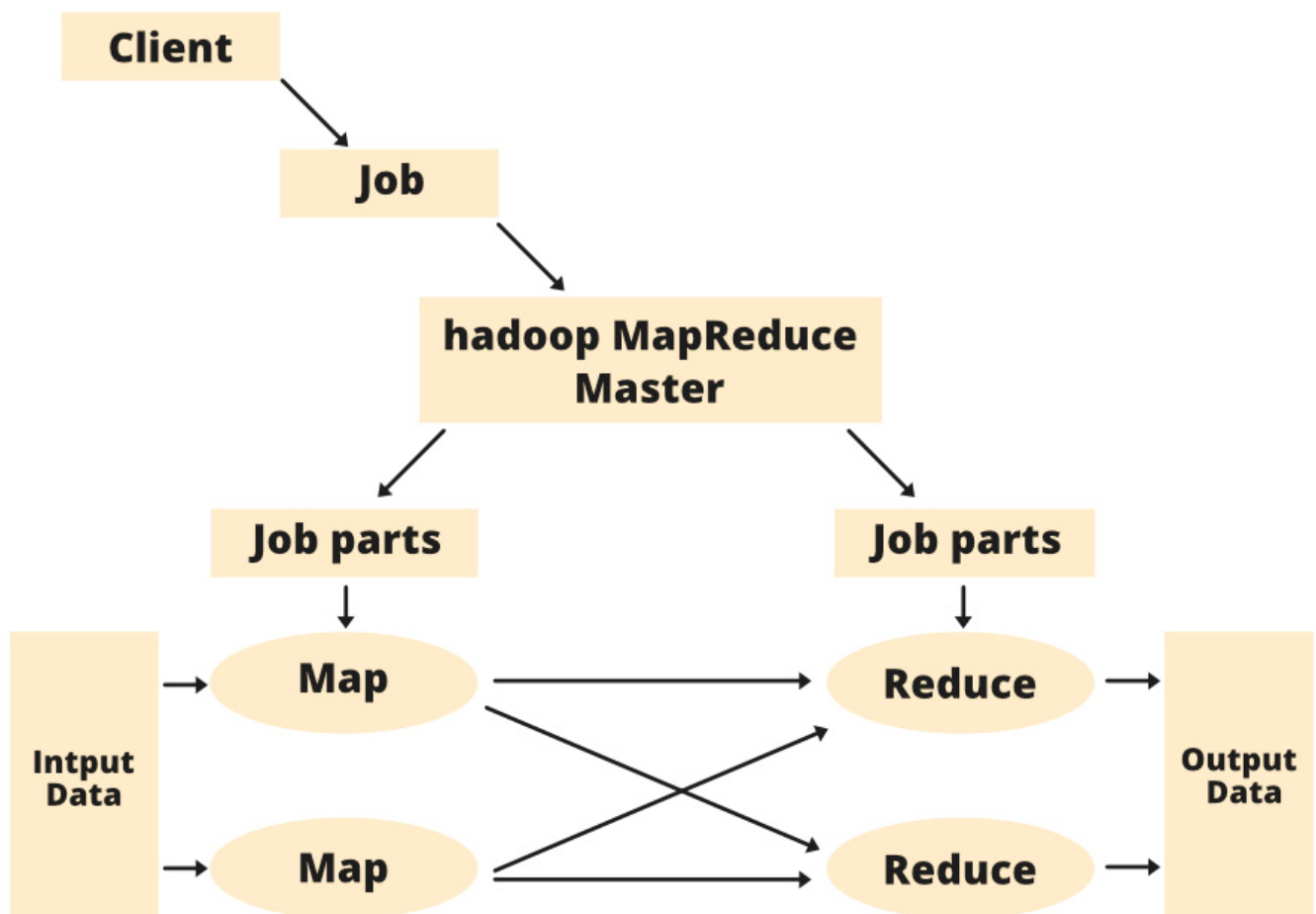
MapReduce Architecture

Last Updated : 04 Aug, 2025



MapReduce Architecture is the backbone of **Hadoop's processing**, offering a framework that **splits jobs into smaller tasks**, **executes** them in parallel across a cluster, and **merges results**. Its design ensures parallelism, data locality, fault tolerance, and scalability, making it ideal for applications like log analysis, indexing, machine learning, and recommendation systems.

Map Reduce Architecture



Core Components of MapReduce Architecture

The MapReduce Architecture follows a **master-slave model**, where the Job Tracker (**master**) coordinates tasks and **Task Trackers (slaves)** execute them on cluster nodes. Below are its main components:

1. Client

The Client is the entry point into MapReduce. It submits the job for execution by packaging the Mapper and Reducer logic into a JAR file and specifying the input and output paths. After submission, the Client's role ends.

2. Job

A Job represents the complete processing request from the client.

- Internally, it is divided into job parts (tasks).
- Each job part is assigned to either the Map or Reduce phase.

3. Hadoop MapReduce Master

The Master Node coordinates job execution. It:

- Accepts jobs from the Client.
- Splits them into job parts.
- Assigns these parts to the Map and Reduce tasks.
- Tracks execution progress and reassigns failed tasks.

4. Job Parts (Tasks)

Every job is divided into **smaller units called job parts**.

- Map job parts: Process input data splits into intermediate key-value pairs.
- Reduce job parts: Aggregate intermediate results into final outputs.

5. Map Phase

- Input data is split and given to Map tasks.
- Each Map task processes its local split and produces intermediate results in the form of key-value pairs.

6. Shuffle & Sort (Between Map and Reduce)

- Intermediate key-value pairs from the Map phase are grouped by key.
- Sorting ensures ordered keys before passing them to the Reducers.

7. Reduce Phase

- Reducers take grouped keys and values from Shuffle & Sort.
- They aggregate them to produce the final output data, which is written back to HDFS.

Phases in MapReduce Architecture

The MapReduce model processes large datasets in two main phases—Map and Reduce—with an intermediate Shuffle & Sort stage that organizes data between them.

Map Phase

The input dataset is divided into splits, each processed by a Map Task on the node storing the data (ensuring data locality). A `RecordReader` converts raw input into (key, value) pairs, which the Mapper transforms into intermediate results (e.g., "Hello Hadoop" → (Hello, 1), (Hadoop, 1)). The Map Phase generates but does not aggregate data.

Shuffle & Sort Phase

After mapping, the intermediate outputs are reorganized so they can be processed efficiently by reducers. Shuffling groups identical keys, e.g., (Hadoop, 1), (Hadoop, 1), (Hadoop, 1) becomes (Hadoop, [1,1,1]). Sorting then arranges the keys in order. This stage ensures balanced distribution of work and is essential for linking the Map and Reduce phases.

Reduce Phase

Finally, reducers take the grouped keys and their associated values from the Shuffle & Sort stage. The `reduce()` function aggregates or summarizes them to produce the final output. For example, (Hadoop, [1,1,1]) becomes (Hadoop, 3). The consolidated results are written back into HDFS at the client-specified output location.

Execution Flow of MapReduce Architecture

1. Client submits the job includes JAR file, Mapper & Reducer logic, input path and output path.
2. Job Tracker accepts the job, splits it into tasks and assigns them to Task Trackers based on data locality.
3. Task Trackers execute Map Tasks on input splits stored locally.
4. Intermediate results are shuffled and sorted.
5. Reduce Tasks process grouped results and generate the final output.
6. Final output is written back to HDFS.

The design is optimized for data locality, which minimizes network transfer by moving computation close to where the data is stored.

Optimizations in MapReduce Architecture

- **Speculative Execution** : If a task is running unusually slow, Hadoop launches a duplicate copy on another node. The task that finishes first is accepted, improving reliability and efficiency.
- **Fault Tolerance** : If a task or node fails, the Job Tracker reassigns the task to another node that has a replica of the data.
- **Load Balancing** : The Job Tracker distributes tasks across available Task Trackers to utilize the cluster efficiently.
- **Pipeline Execution** : Multiple tasks (Map and Reduce) run concurrently across the cluster, ensuring maximum throughput.

Advantages of MapReduce Architecture

- **Parallelism** : Tasks are executed simultaneously across hundreds or thousands of nodes.
- **Scalability** : Handles increasing data volumes by simply adding more nodes to the cluster.
- **Fault Tolerance** : Automatically recovers from task or node failures using replicated data.
- **Efficiency** : Reduces network traffic by processing data locally on the nodes where it resides.

- **Simplicity** : Developers only need to implement map() and reduce() functions; Hadoop handles distribution, fault tolerance and scheduling.

 Comment

More info 

Advertise with us

Explore

Software Engineering Basics

▼

Software Measurement & Metrics

▼

Software Development Models & Agile Methods

▼

SRS & SPM

▼

Testing & Debugging

▼

Verification & Validation

▼

Practice Questions

▼

Do Not Sell or Share My Personal Information